

GUÍA DE IMPLEMENTACIÓN DE LA CAPA DE DATOS

Ficha de Fuentes de Datos y Plan de Tratamiento ETL

Esta es la tercera y última ficha del curso. Junto con la ficha del caso de uso (Sesión 1) y la ficha de arquitectura (Sesión 2), completa el contrato técnico de tu proyecto: la primera definió qué problema resuelves y por qué, la segunda con qué lo construyes, y esta define de dónde vienen tus datos y qué le harás a cada fuente para que llegue limpia y trazable hasta tu modelo. Cuando las tres estén resueltas, la Sesión 4 deja de ser teoría: empezamos a escribir código.

Las tres capas, en una frase

Todo lo que decidas aquí empuja tus datos por tres capas. La capa Raw (Bronze) guarda una copia fiel e inmutable de la fuente, sin tocar nada. La capa Staging (Silver) contiene esos mismos datos ya parseados, limpios y validados contra un esquema estricto. La capa Curated (Gold) entrega el dato listo para el modelo: fragmentos, embeddings y metadatos. La regla de oro es que cada capa solo lee de la anterior y escribe sobre sí misma; nunca se altera Raw ni se modifica Staging desde Curated.

Cómo completar esta ficha

Trabaja una fuente de datos a la vez. En la columna celeste de cada fila encontrarás el campo, una breve explicación de lo que se espera y un ejemplo resuelto sobre el caso CRM de cotizaciones que venimos usando en clase. Escribe tu propia respuesta en el recuadro de la derecha. Si tu caso tiene varias fuentes, vuelve a llenar la ficha una vez por cada una. Una ficha incompleta no es mala: solo indica qué decisión de pipeline aún no has tomado.

IDENTIDAD DE LA FUENTE	
Fuente (qué es / por qué) Describe qué es la fuente y por qué tu caso de uso la necesita. <i>Ejemplo:</i> Bandeja de correo comercial; cada correo trae 1 a 3 cotizaciones de proveedores en PDF que hoy se transcriben a mano al CRM.	Registros de gastos de producción de libros. Necesarios para consolidar el costo real (edición, imprenta, derechos) y permitir que el agente sugiera un precio de venta basado en márgenes dinámicos.
Tipo y método de acceso Indica si es API, archivos, correos o base de datos, y cómo te conectas. <i>Ejemplo:</i> API de correo (p. ej. Gmail API) vía OAuth, con descarga de los adjuntos PDF.	Base de datos relacional (SQL). Acceso mediante consulta directa a una Materialized View (view_gastos_proyecto) en PostgreSQL, lo que asegura una vista consolidada de los gastos registrados.
Volumen y frecuencia Cuántos registros y cada cuánto; aquí se justifican el costo y la resiliencia. <i>Ejemplo:</i> ~80 correos por semana, con picos los lunes. Objetivos: < 5 min por ficha, p95 < 30 s, costo < US\$0,15.	~10-15 materiales por semana. El sistema se activa bajo demanda (trigger) al cerrar el registro de gastos. Objetivos: Ciclo completo < 15 min, latencia < 60s y costo < \$0.50 USD.
EXTRACCIÓN (E)	

Idempotencia (ID estable) Qué identificador derivado del contenido evitará duplicados al reprocesar. <i>Ejemplo: ID = message_id del correo + SHA-256 del adjunto; siempre upsert, nunca insert ciego.</i>	El identificador estable será el idproyecto. Al reprocesar un cálculo, se realizará un upsert en la tabla de logs de inferencia para evitar duplicar propuestas de precio para un mismo material.
Patrón de resiliencia Retry con backoff, circuit breaker o checkpoints, según la fragilidad de la fuente. <i>Ejemplo: Retry con backoff exponencial (la API da 429 en los picos) + checkpoint por lote procesado.</i>	Retry con backoff exponencial para la conexión SQL y un refresco de la vista disparado por evento (webhook) cada vez que se registre un gasto crítico, asegurando frescura de datos.
Metadatos de extracción Qué registrarás del acto de extraer, separado del contenido en sí. <i>Ejemplo: Timestamp, remitente, message_id, hash del archivo y ruta en el Object Store (capa Raw).</i>	Se registrará: idproyecto, timestamp_extraccion, version_esquema_db y el run_id de la corrida del pipeline para garantizar trazabilidad bidireccional.
TRANSFORMACIÓN (T)	
Ruta de parseo Qué señal temprana decide entre un parser barato y el OCR de nube. <i>Ejemplo: Si el PDF tiene texto (caracteres > 500) va a pdfplumber; si está escaneado, a Document AI / Textract.</i>	Al ser datos estructurados en SQL, no se requiere OCR. El parseo consiste en la validación de tipos vía Pydantic para asegurar que los montos sean numéricos y las fechas válidas antes de enviarlas al modelo.
Limpieza de ruido Qué ruido propio de tu dominio hay que remover. <i>Ejemplo: Quitar firmas, disclaimers y encabezados/pies repetidos; normalizar Unicode a forma NFKC.</i>	Normalización Unicode (forma NFKC) en las descripciones de los gastos para eliminar caracteres invisibles y limpieza de espacios en blanco en las claves de producto (claveproducto).
Validación (esquema) Qué campos y tipos valida tu esquema antes de pasar a Staging. <i>Ejemplo: Pydantic {proveedor, producto, cantidad, precio_unitario, moneda, validez}; confianza < 0,9 queda nulo, nunca inventar.</i>	Esquema Pydantic estricto: {idproyecto, listado_gastos: List[float], politica_margen: float}. Si el agente detecta inconsistencias (confianza < 0.8), el campo precio se devuelve como null.
Chunking Fijo, por estructura o semántico; justifícalo por costo y contexto. <i>Ejemplo: Por estructura: cada ítem de la cotización es un bloque; preserva mejor el contexto.</i>	No aplica chunking semántico. Se procesa el listado completo de gastos como un solo bloque de contexto por proyecto para garantizar que el agente razone sobre la integridad del costo total.
CARGA (L)	
Destino(s) Relacional, vector store o ambos. <i>Ejemplo: Base relacional (campos para el CRM) + vector store (texto del correo para búsqueda semántica).</i>	Base Relacional (PostgreSQL). Los resultados se cargan en una tabla de Logs de Inferencia para auditoría y en la tabla operativa de productos para actualizar el precio sugerido.

ID determinístico Cómo compones el identificador para auditar de dónde salió cada fragmento. <i>Ejemplo:</i> DocID(message_id) :: ChunkIndex :: ContentHash.	Se compondrá como: idproyecto :: timestamp_ejecucion :: version_modelo. Esto permite saber exactamente qué versión del agente y qué datos originaron cada precio.
Escritura atómica Cómo evitas registros huérfanos entre los dos destinos. <i>Ejemplo:</i> Relacional en PENDING → embeddings → marcar READY; un job de reconciliación limpia los PENDING colgados.	Se inserta el registro en estado PENDING al iniciar la llamada al LLM; una vez que el JSON de salida es validado y persistido, se actualiza el estado a READY.
ROBUSTEZ Y OBSERVABILIDAD	
Criterio de DLQ Qué se desvía a la cola de errores y con qué contexto. <i>Ejemplo:</i> PDF corrupto, encoding ilegible o validación fallida → DLQ con etapa + error + traceback + run_id.	Proyectos con gastos en cero, materiales sin clave válida en el catálogo maestro o fallas de validación de tipos financieras se desvían a la Dead Letter Queue para revisión manual.
Métricas a vigilar Qué medirás para no quedar ciego en producción. <i>Ejemplo:</i> Docs/hora, latencia p95 por etapa, % desviado a la DLQ y tamaño acumulado de la DLQ.	Tiempo total del ciclo (Costo + Precio), latencia p95, tasa de aceptación del precio sugerido (> 85%) y porcentaje de registros en la DLQ.
AUTODIAGNÓSTICO	
Pecado más probable + mitigación De los 5 pecados del ETL en IA, cuál te amenaza más aquí y cómo lo evitas. <i>Ejemplo:</i> Parseo de fuerza bruta (mandar todo al OCR quiebra el presupuesto); mitigación: el enrutador reserva el OCR solo para escaneados.	Datos sucios e inconsistencia en gastos (registros manuales ambiguos). Mitigación: Implementación de un filtro de confianza estricto (> 0.8); si hay duda, se etiqueta el campo como nulo y se alerta al analista.

Antes de pasar a la Clase 4

Tu capa de datos está lista para convertirse en código cuando puedes responder que sí a todo lo siguiente:

1. Cada fuente de tu caso tiene su propia ficha completada.
2. Para cada fuente sabes su ID estable y cómo garantizas la idempotencia.
3. Tienes definidas la ruta de parseo y el esquema de validación que llevan el dato a Staging.
4. Sabes a qué capas escribe y cómo mantienes la escritura atómica.
5. Tienes claro qué se desvía a la DLQ y qué métricas vigilarás.
6. Identificaste el pecado capital más probable y su mitigación.

Un pipeline bien hecho hoy es ejecución táctica mañana. Con estas tres fichas resueltas, la Clase 4 ya no diseña: construye.